
Lab 4

Advanced Data Manipulation

CSE 4308
DATABASE MANAGEMENT SYSTEMS LAB

AUGUST 30, 2023

Contents

1	Distinct	2
2	Range Operator	2
2.1	BETWEEN Operator	2
2.2	IN Operator	2
3	Rename	3
4	String Operator	3
4.1	LIKE Operator	3
4.2	Concatination	3
5	Data Sorting	4
6	Aggregation Function	4
6.1	Group By	4
6.2	Having	5
7	Subquery	5
8	Task	6

1 Distinct

The DISTINCT statement is used to return only distinct (different) values. Inside a table, a column often contains many duplicate values; and sometimes you only want to list the different (distinct) values. The syntax is following:

```
SELECT DISTINCT attributename  
FROM tablename;
```

For example,

```
SELECT DISTINCT DEPT_NAME FROM EMPLOYEE;
```

2 Range Operator

2.1 BETWEEN Operator

The BETWEEN operator selects values within a given range. The values can be numbers, strings or dates. The BETWEEN operator is inclusive: begin and end values will be included. However, you have to provide the lower value first then the upper value.

```
SELECT attribute_1, .....  
FROM tablename  
WHERE attribute_1 BETWEEN l_value AND u_value;
```

For example,

```
SELECT NAME, DEPT_NAME, SALARY  
FROM EMPLOYEE  
WHERE SALARY BETWEEN 35000 AND 80000;
```

For excluding a range we can use NOT operator before BETWEEN. For example,

```
SELECT NAME, DEPT_NAME, SALARY  
FROM EMPLOYEE  
WHERE SALARY NOT BETWEEN 35000 AND 80000;
```

2.2 IN Operator

The IN operator allows one to specify multiple values in WHERE clause. It is a shorthand for multiple OR conditions.

```
SELECT attribute_1, .....  
FROM tablename  
WHERE attribute_1 IN (value1, value2, ...);
```

For example,

```
SELECT NAME, SALARY  
FROM EMPLOYEE  
WHERE DEPT_NAME in ('DEV', 'TESTING');
```

Similar to between, we can use NOT to exclude some values.

3 Rename

SQL aliases are used to give a table, or a column in a table, a temporary name. We can also use it for renaming an expression or an entire query. The primary purpose of using aliases is to make a column or table more readable. For example:

For example,

```
SELECT ID, NAME, (SALARY*12) AS Y_SALARY
FROM EMPLOYEE;
```

```
SELECT E.NAME, P.NAME
FROM EMPLOYEE E, (SELECT * FROM PROJECT) P
WHERE E.P_ID=P.P_ID;
```

Here using AS is optional for the expression, table and any query.

4 String Operator

4.1 LIKE Operator

In simple terms, the LIKE operator is used to match substrings in a query. It is used in a WHERE clause to search for a specified pattern in a column. There are two wildcards often used in conjunction with the LIKE operator:

- % to represent any substring.
- _ to represent any single character.

```
SELECT attribute_1, attribute_2 .....
FROM tablename
WHERE attribute_1 LIKE pattern;
```

Here, Patterns are case-sensitive. For example,

```
SELECT NAME
FROM EMPLOYEE
WHERE NAME LIKE 'A%';
```

So, It will show you all the names that start with capital A. For restricting the name size of the resultant name additionally, let's say 4 characters. We can use the following statement:

```
SELECT NAME
FROM EMPLOYEE
WHERE NAME LIKE 'A___';
```

4.2 Concatination

To concatenate multiple columns or any additional string with any column at the time of retrieving data, SQL supports string operator(||). For instance,

```
SELECT 'Employee NAME: ' || NAME
FROM EMPLOYEE;
```

It will show the term 'Employee NAME: ' as the prefix of any name.

5 Data Sorting

The ORDER BY keyword is used to sort the result-set in ascending or descending order. If one does not specify the sorting order, the ORDER BY keyword sorts the records in ascending order by default. Otherwise, to specify ascending or descending order of sorting, we use the keywords 'ASC' and 'DESC' respectively.

```
SELECT attribute_1, attribute_2 .....  
FROM tablename  
ORDER BY attribute_1 [ASC/DESC], .....;
```

For example,

```
SELECT NAME , SALARY  
FROM EMPLOYEE  
ORDER BY DEPT , SALARY DESC;
```

6 Aggregation Function

Aggregate functions are functions that take a collection (a set or multiset) of values as input and return a single value. SQL offers five built-in aggregate functions:

- Average: avg
- Minimum: min
- Maximum: max
- Total: sum
- Count: count

The input to sum and avg must be a collection of numbers, but the other operators can operate on collections of non-numeric data types, such as strings, as well. The syntax is following:

```
SELECT [avg/min/max/sum/count](attribute_1)  
FROM EMPLOYEE;
```

For example,

```
SELECT avg(SALARY)  
FROM EMPLOYEE;
```

6.1 Group By

There are circumstances where we would like to apply the aggregate function not only to a single set of tuples but also to a group of sets of tuples; we specify this wish in SQL using the GROUP BY clause. The attribute or attributes given in the group by clause are used to form groups. Let's say, now we want to know the average salary of each department. So, the statement will be:

```
SELECT DEPT , avg(SALARY)  
FROM EMPLOYEE  
GROUP BY DEPT;
```

All selected attributes must be present either in aggregate functions or in group by clause, otherwise the query will be considered erroneous. For example, the following query is incorrect as ID appears in the SELECT statement, but not in the GROUP BY clause:

```
SELECT DEPT , ID , avg(SALARY)  
FROM EMPLOYEE  
GROUP BY DEPT;
```

6.2 Having

The HAVING clause is used to specify conditions on groups rather than on tuples. For example, we might be interested in only those departments where the average salary of the employees is more than \$5000. We cannot use the WHERE clause in this case because WHERE is used to specify conditions on a single set of tuples, whereas in this case, we are interested in finding a group of departments where the average salary is more than \$5000.

```
SELECT DEPT , avg(SALARY)
FROM EMPLOYEE
GROUP BY DEPT
HAVING avg(SALARY)>5000;
```

As was the case for the SELECT statement, any attribute that is present in the HAVING clause without being aggregated must appear in the GROUP BY clause, otherwise the query is treated as erroneous.

7 Subquery

SQL provides a mechanism for nesting subqueries. A subquery is a select-from-where expression that is nested within another query. It can be placed at two places:

- In the WHERE clause of a SELECT statement. Example,

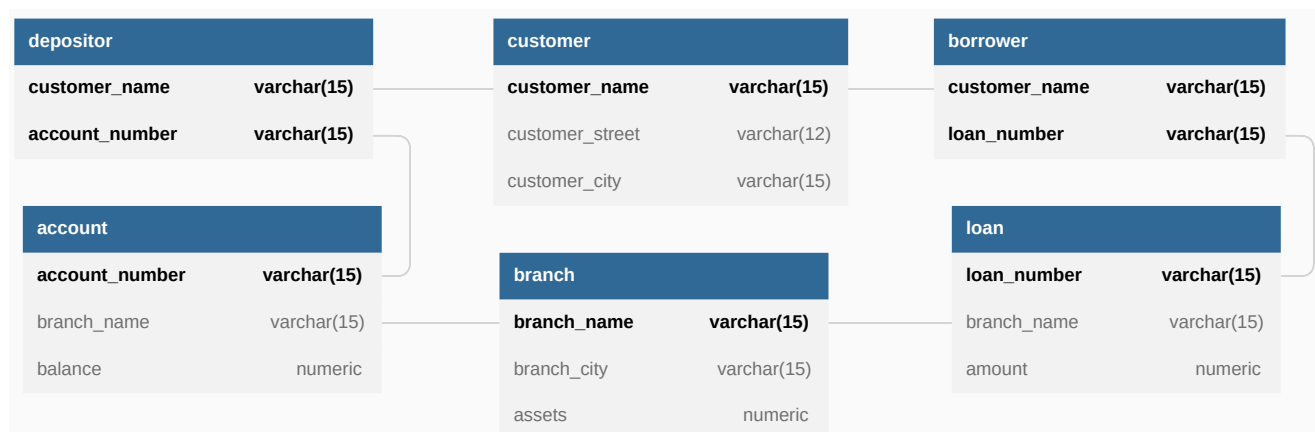
```
SELECT DEPT
FROM EMPLOYEE
WHERE SALARY >
      (SELECT avg(SALARY)
       FROM EMPLOYEE);
```

- In the FROM clause of a SELECT statement. The same result can be produced by using subquery in FROM clause as following:

```
SELECT DEPT
FROM EMPLOYEE E, (SELECT avg(SALARY) as A_SALARY
                  FROM EMPLOYEE) A
WHERE E.SALARY > A.A_SALARY;
```

8 Task

Execute the `banking.sql` script using `command`. It creates a set of tables along with values that maintain the following schema:



Here, the boldfaces denote the primary keys and the arcs denote the foreign key relationships. In this lab, you have to write all SQL statements in an editor first and save them with `.sql` extension. Then execute the SQL script.

Write SQL statements for the following queries:

- Find all the `customer_city` that ends with 'd' and have a letter 'f' somewhere in the string.
- Find all customer names and their cities who have a loan but not an account.
- Find all customer-related information about who has an account or a loan.
- Find the total assets of all branches.
- Find the name of the branch city and the total number of accounts at each branch city.
- Find the customer name and account number of the account that has the highest balance.
- Find the average balance of accounts at each branch and display them according to their branch name in ascending order and the average balance in descending order.
- Find the number of customers of each `customer_city` who are depositors as well as have loans.
- Find the average loan amount at each branch. Do not include any branch which is located in a that has the substring, 'Horse' in its name.
- Find all customer-related information who have an account in a branch, located in the same city as they live.
- Find those branch names that have a total account balance greater than the average total balance among all the branches.
- Find those branch names where the total customer balance is greater than the total customer loan amount.
- Find the names of the customers who have at least one loan that can be paid off by his/her total balance.
- Find those customers' names, balances and loan amounts who have accounts and loans but neither in the branch of their own city.
- Find the customers' names who have accounts or loans in the branch that has the highest assets.